2021 IEEE Global Conference on Artificial Intelligence and Internet of Things (GCAIoT) | 978-1-6654-3841-4/21/$31.00 ©2021 IEEE | DOI: 10.1109/GCAIoT53516.2021.9693046

Asynchronous Coarse-Grained Load Migration Scheme for IoT Applications in Fog Networks

Mohammed A. Jasim[1], Nazli Siasi[2], Mohammad Almalag[2], Vahraz Honary[1], Adel Aldalbahi[3]
[1]School of Engineering, University of Mount Union, Alliance, OH, USA
[2]Department of Physics, Computer Science and Engineering, Christopher Newport University, Newport News, VA, USA
[3]Department of Electrical Engineering, King Faisal University, Hofuf, KSA
Email: {jaesimad, honaryva}@mountunion.edu, {nazli.siasi, mohammad.almalag}@cnu.edu, aaldalbahi@kfu.edu.sa

***Abstract*—Fog computing provides distributed processing and storage solutions for real-time applications that demand low latency and fast response times. This makes fog solutions suitable for *internet-of-things* (IoT) devices that request various network functions. To offer multiple functions for IoT applications, fog nodes can leverage *network function virtualization* (NFV) for a scalable and elastic function modification, i.e., without the need for dedicated hardware. Despite the saliencies achieved from the synergistic combination between fog and NFV technologies, a key challenge here is the limited resources at the fog nodes. This makes the latter susceptible to rapid node saturation and network congestion at high traffic volumes. Along this, efficient resource utilization and load distribution mechanisms are necessary to enhance admission rates and *quality-of-service* (QoS). Hence, this paper proposes novel load migration schemes for NFV-based fog networks that aim to reduce the overhead of the migration process. Namely, a coarse-grained load diffusion scheme is adopted to reduce migration frequencies, incurred delay, and cost. Further, destination nodes are selected based on *least-load* (LL) or *least-delay* (LD) mechanisms to accommodate delay-sensitive, delay-tolerant, and computation-intensive IoT applications.**

***Index Terms*— Coarse-grained, fog computing, least-delay, least-load, load migration, migration frequency, network function virtualization**

I. Introduction

FOG computing first proposed in [1] brings cloud services to the edge of the network at a closer proximity to the source of data (end-user terminals). This allows local data processing at reduced latency levels without sending them back to the cloud core. Along this, fog paradigms serve delay-sensitive IoT applications that demand rapid service instantiation at minimal control-plane delays. The motivation for fog computing is to reduce the aggregated delays in cloud domains due to the extended propagation distances over geographically dispersed datacenters. Fog computing provides reduced link delays, thus reducing latency and improving response times. It also enhances services for remote locations to enable data computation and storage, where access to the cloud core is limited. Further, the local processing of data reduces cost of bandwidth and alleviates traffic at the cloud domain. However, one limitation of fog computing is the limited resources. Consequently, fog nodes are vulnerable to network congestion and rapid saturation during high traffic volumes, particularly when terminals demand intensive data computation for prolonged durations. This exhausts network resources, degrades the QoS, and reduces capacity and revenue. Hence, efficient resource management is vital for fog computing for enhanced network utilization.

As mentioned earlier, the distributed fog nodes aim to provide rapid service provisioning. However, one challenge that can reduce network response time is the instantiation of network functions on the fog servers, where these functions vary with the user demands. Hence, dedicated hardware adds a burden to the operation of fog nodes. Along this, it is beneficial to provide elastic and dynamic function embedding on the fog servers, where the type of functions installed on the hardware vary according to the user demands. Hence, *network function virtualization* (NFV) proposed in [2] is leveraged here for fog computing to realize this purpose. NFV uses general-purpose commodity servers to install modular software, thus it separates network functions from dedicated hardware to achieve elastic, efficient, and scalable services. This separation reduces operational and maintenance costs for network operators, along with rapid services to users regardless of the requested services and location of the servers, as each server now can dynamically change the offered functions at faster deployment times. Software applications are provided through the *virtual network functions* (VNFs) that deliver the specific network functions such as security functions (firewalls, intrusion detection, etc.), IP multimedia subsystems, *evolved packet core* (EPC), network slicing, virtual customer premises equipment, and content delivery networks. Further, the VNFs here are instantiated dependently, termed as *service function chaining* (SFC). Hence the mapping of VNFs on the host fog nodes form a sequential path over which traffic transverses. Along this, SFC provisioning is an essential process in NVF-based fog networks as studied in [3]-[6], while achieving efficient operation under congestion and saturation conditions. Therefore, achieving load balancing and migration enhances the network performance and utilization rates.

Many studies have been proposed for load management in fog computing that range from dynamic, static to opportunistic strategies using heuristics, optimization, and neural network tools, see [7]-[13] surveyed in Section II. Overall, these efforts often implement limited tasks (e.g., VNFs), suffer from

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 08:42:04 UTC from IEEE Xplore. Restrictions apply.

redundant traffic, lack analysis of migration frequency and cost. In addition, existing schemes do not consider NFV-based fog servers with SFC provisioning.

In light of the above, this paper provides a novel load migration scheme for NFV-based fog computing for various SFC demands. The network operates in a realistic setting, where limited resources are adopted in a fog-cloud architecture to account for node congestion and network saturation. The goal is to propose an efficient load migration scheme that diffuses the excess load (VNFs) from overutilized nodes that are approaching saturation to less utilized nodes. Hence, overutilized nodes are relieved and node saturation rate is reduced. The migration process adopts a coarse-grained approach to achieve reduced migration frequency, minimized delay and cost. The approach selects the destination nodes that can host the excess load based on *least-load* (LL) and *least-delay* (LD) migration schemes that are suitable for delay-sensitive, delay-tolerant and computation-intensive requests.

This paper is organized as follows. First, Section II surveys existing work on load migration in fog computing. Section III presents the network model and architecture. This is followed by the proposed load migration scheme in Section IV. Performance evaluation and results in Section V, along with conclusions in Section VI.

II. Related Work

Multiple studies have investigated load migration in fog computing. First, a graph repartitioning approach is presented in [7] to achieve a dynamic load balancing mechanism at the benefit of increased configuration and reduced consumption and migration frequency. However, the work is limited to 10 tasks only and lacks a feasible study on wide-scale traffic volumes. The work in [8] achieves load balancing using a breadth-first search algorithm, along with an authentication mechanism for 25 tasks only. However, the work lacks service response times. Also, the exchange of control messaging here increases redundant traffic and reduces the overall network capacity, along with high delays. Moreover, the static nature of the load broadcasting inquiries yields in request blockages, as the nodes wait until the overloading status has reached before the broadcasting process starts.

The work in [9] proposes an energy-aware load balancing and scheduling method using an enhanced version of the particle swarm optimization algorithm. However, this work focuses on analyzing the workload and time in smart factory settings only and lacks a comprehensive analysis. The work in [10] uses Lyapunov optimization to perform joint tasks and energy online offloading from one fog node to multiple neighboring helper nodes to minimize task execution delay, as well as the energy consumption for a specific task. Another offloading policy to neighboring helper nodes is introduced in [11], where nodes share their computing resources to enhance the energy efficiency in the network. An opportunistic placement algorithm based on neural networks is leveraged in a hierarchical architecture to determine host nodes in [12]. The algorithm balances the load based on computation and communication delays. Note that placing loads at the upper-tier yields longer response times. Based on the load status, edge-edge migration modes are supported in [13] in a fog architecture. It is composed of host nodes and registration (management) nodes responsible for load detection and offloading. Overall, the work also lacks a feasibility study and simulation models.

III. Network Architecture and Model

A. Network Architecture

A fog-cloud paradigm is adopted in this work to align with the proposed OpenFog reference architecture in [14], as depicted in Figure 1. Consider the following entities that comprise the different layers in the architecture.

Terminals (Layer 1): End-users terminals that demand various applications and services of different delay and capacity requirements. Terminals here include mobile stations, sensors, vehicles, desktops, laptops, etc. Terminals are distributed uniformly across the fog nodes in each cluster, where each cluster is composed of multiple nodes.

Fog Nodes (Layer 2): This layer consists of distributed fog nodes at the proximity of users in Layer 1 and cloud nodes in Layer 3. It acts as the gateway that receives traffic requests and provides service instantiation while meeting latency and bandwidth demands for various applications. The fog nodes possess limited computing and storage resources, at the advantage of reduced link delays. All the fog nodes here are homogenous and are interconnected with each other via direct links.

Cloud Core (Layer 3): It consists of a few dispersed datacenters of high-capacity nodes that are interconnected directly via high bandwidth links. The cloud core is also interconnected with the fog nodes in Layer 2, where it places network functions on the fog nodes, as per the user demands. Note that service embedding is not conducted on the cloud core due to the relaxed delay bounds as compared to fog domains. Further, extending service embedding on the cloud domain can follow a similar approach.

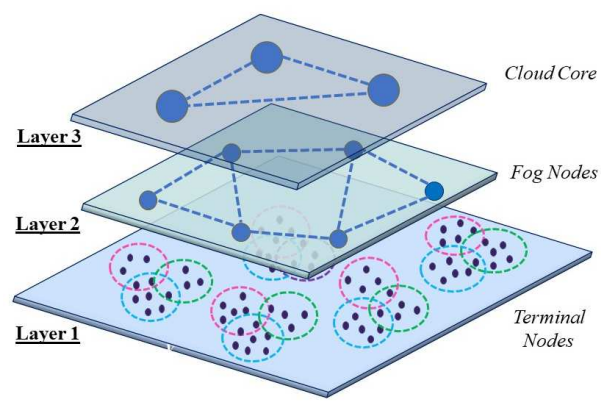

Fig. 1. Fog-Cloud Architecture

B. Network Model

The substrate network for the fog architecture is modeled as an undirected graph $\mathbf{G} = (\mathbf{N}, \mathbf{E})$, where $\mathbf{N}$ and $\mathbf{E}$ denote in order the set of nodes and links. First, the set $\mathbf{N}=\{n_j^i\}$ is comprised of all terminals, fog, and cloud nodes. In notations, n_j^i is the i-th node in Layer j, i.e., $j = 1$ for terminals, $j = 2$ for fog nodes or $j = 3$ for cloud nodes. Nodes in Layers 2 and 3 possess processing capacity $q_{prc}(n_j^i)$ and memory $q_{me}(n_j^i)$, i.e., $q_{prc}(n_j^i) \leq Q_{prc}(n_j^i)$ and $q_{me}(n_j^i) \leq Q_{me}(n_j^i)$, where $Q_{prc}(n_j^i)$ and $Q_{me}(n_j^i)$ are the

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 08:42:04 UTC from IEEE Xplore. Restrictions apply.

maximum capacity and memory, respectively. Here upper layer possesses higher resources, e.g., $Q_{prc}(n_3^i) > Q_{prc}(n_2^i)$ and $Q_{me}(n_3^i) > Q_{me}(n_2^i)$. Also, incurred processing delay at node n_j^i is denoted by $D_{prc}(n_j^i)$, and cost is expressed by $C(n_j^i)$. Further, the set $\mathbf{E} = \{e_{jl}^{ih}\}$ consists of links in the network that interconnects various layers, i.e., wireless links between Layers 1 and 2 and wireline between fog and cloud layers. Here e_{kl}^{ih} is the link that interconnects the i-th node at Layer j with the h-th node at Layer l. The instantaneous available link bandwidth is denoted by $b_e(n_j^i, n_l^h)$, where $b_e \le B_e$. Here B_e denotes the maximum available free bandwidth. Further, the propagation delay and cost at the link is expressed by $D_{prp}e(n_j^i, n_l^h)$ and $C(e)$, respectively.

C. Terminal Request Model

A set of terminal nodes generate a batch of requests $\mathbf{R}=\{r\}$ that vary in terms of service deadline, computing resources, and service type. The request is received by a fog node in Layer 2, termed as the source node, *src*. The request then specifies the destination node *dst* for the traffic, i.e., *src*, *dst* $\in \mathbf{N}$, which can be the same terminal that generates request r another terminal node that is close to a different fog node. A path p is established between the *src* and *dst* at which the VNFs for request r are mapped on the intermediate fog nodes. The latter must possess sufficient resources that accommodate the demands for the VNFs in terms of computing and memory resources. Along this, each request can be modeled by a 6-tuple $r = < src, dst, \mathbf{F}_r, Q_r, b_r, \delta_r >$. The variable $\mathbf{F}_r$ defines the set of dependent VNFs that form the SFC, i.e., embedded on a host fog node $\breve{n}_j^i \in \breve{\mathbf{N}}$, i.e., $\mathbf{F}_r=\{f_t | t=1, 2, \ldots, T\}$. The set is composed of $t \in T$ VNF types, e.g., $\mathbf{F}_r = \{f_1 \rightarrow f_2 \rightarrow f_3 \rightarrow f_4\}$, where T is the total number of VNF types. Furthermore, each request specifies nodes and links resources determined by Q_r and b_r, respectively. Here Q_r accounts for the total processing and memory requirements per each VNF, $q_{prc}(f_t)$ and $q_{me}(f_t)$, respectively, i.e.,

$$Q_r = \sum_{\forall f_t \in \mathbf{F}_r} q_{prc}(f_t) + \sum_{\forall f_t \in \mathbf{F}_r} q_{me}(f_t). \tag{1}$$

Finally, the variable δ_r limits the time over which a service is embedded before the start of the data-plane, where if this bound is exceeded, then the request is dropped from the network and is considered as failed, thus degrading *quality-of-experience* (QoE) for the terminal.

IV. Coarse-Grained Load Migration Scheme

Consider that the instantaneous load at any fog node is denoted by $L_{ins}(n_2^i)$, where it hosts different VNFs of multiple requests. If $L_{ins}(n_2^i)$ exceeds a threshold load $L_{th}(n_2^i)$, this indicates an overutilized node that can approach saturation. This in turn makes it vulnerable to failure and requests blockage. Along this, load migration is necessary to shift the load from overutilized to underutilized nodes in the network. First, the overutilized node determines the amount of excess load at the node $L_{exc}(n_2^i)$ that need to be migrated, i.e.,

$$L_{exc}(n_2^i) = \begin{cases} L_{ins}(n_2^i) - L_{avg}(n_2^i), & \text{if } L_{ins}(n_2^i) > L_{avg}(n_2^i), \forall n_2^i \in \mathbf{N}, \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

The amount of migrated load from the overutilized node is determined by the excess load. In general, this load is portioned and diffused among other neighboring nodes to achieve load balancing. The increased level of load partitioning creates more distributed loads in the network, which in turn increases communication overhead, synchronization, and delay that aggregates to the SFC provisioning delay. Further, the distributed load can create redundant traffic that occupies the links and thus compete on limited resources with the incoming requests. Consequently, this reduces network capacity to accommodate more requests. Along this, the load is only partitioned into a few segments, i.e., coarse-grained. This approach considers the average load and utilization level at the node. Hence load migration occurs in a few steps without occupying many links and nodes, i.e., reduced migration frequencies. Note that the goal here aims to alleviate overutilized nodes that are approaching saturation and may cause network blockage for any further incoming requests. Hence the load is migrated to regions with less load in a few steps. Consider the details.

The overutilized node $\breve{n}_2^i$ identifies direct neighboring nodes in its proximity over a single shared link, termed as *first tier* neighbors, i.e., $\mathbf{M_1} = \{n_2^i | d(\breve{n}_2^i, n_2^i) = 1\}$. Next, the list is pruned based on available load as compared to the excess load. Namely, each neighboring node $n_2^i \in \mathbf{M_1}$ gauges the available load level, $L_{aval}(n_2^i) = \sigma L_{max}(n_2^i) - L_{ins}(n_2^i)$, it then sends it to $\breve{n}_2^i$. Here σ represents a utilization threshold that maintains a marginal unutilized processing capacity to avoid saturation. Also, the variable $L_{max}(n_2^i)$ is the maximum load that can be accommodated after which the node is considered saturated. Along this, the overall available load at the first neighboring tier must exceed the excess load, i.e.,

$$\sum_{n_2^i \in \mathbf{M}_1} L_{aval}(n_2^i) \ge L_{exc}(\breve{n}_2^i). \tag{2}$$

Alternatively, if all the nodes in the first-tier have insufficient capacity, the overutilized node $\breve{n}_2^i$ extends the neighborhood to the second tier $\mathbf{M_2}$ to find underutilized nodes that can accept excess loads,

$$\mathbf{M_2} = \{n_2^i | d(\breve{n}_2^i, n_2^i) = 2\}. \tag{3}$$

For any set of nodes in the first or second tier, $\breve{n}_2^i$ next determines the amount of migrated load from the total excess load to $\mathbf{M_1}$ or $\mathbf{M_2}$ as per the available resources. Namely, it sorts the neighboring nodes in ascending order based on two schemes, *least-load* (LL) and *least-delay* (LD). First, the LL scheme sorts the neighboring nodes based on their least occupied resources, i.e.,

$$\mathbf{M_1} \rightarrow \mathbf{M_1^{ll}} = \{n_2^i | L_{aval}(n_2^{i+1}) \ge L_{aval}(n_2^{i+1})\}. \tag{4}$$

Hence, the first nodes in the tier possess more resources, which allows them to accommodate the whole or partial excess load. Hence, this approach can diffuse the load in a few steps. Meanwhile, the LD scheme resorts the neighboring nodes based on the least path delay D between $\breve{n}_2^i$ and $n_2^i \in \mathbf{M_1}$ or $\mathbf{M_2}$,

$$\mathbf{M_1} \rightarrow \mathbf{M_1^{ld}} = \{\breve{n}_2^i | D(\breve{n}_2^i, n_2^i) \le D(\breve{n}_2^i, n_2^{i+1})\}, \tag{5}$$

where $D(\breve{n}_2^i, n_2^i) = D_{prp}e(\breve{n}_2^i, n_2^i) + D_{prc}(n_2^i)$.

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 08:42:04 UTC from IEEE Xplore. Restrictions apply.

If the available load at the first node in the LL or LD lists possesses enough resources, $L_{aval}\left(\mathbf{M}_1^{ld}\{1\}\right) \geq L_{exc}\left(\breve{n}_2^i\right)$ then the excess load is migrated in one entity (coarse-grained), i.e.,

$$L_{mig}\left(\breve{n}_2^i \rightarrow \mathbf{M}_1^{ld}\{1\}\right) = L_{exc}\left(\breve{n}_2^i\right). \tag{6}$$

Input: $\mathbf{G} = (\mathbf{N}, \mathbf{E})$, $\mathbf{R}_{map}$, $\breve{\mathbf{N}} = \{\breve{n}_2^i\}$, $Q_{prc}(n_2^i)$, $Q_{me}(n_2^i)$, $b_e(n_2^i, n_2^{i+1})$, $r = < src, dst, \mathbf{F}_r, Q_r, b_r, \delta_r >$

Output: $L_{ins}(n_2^i) \leq L_{th}(n_2^i)\ \forall n_2^i \in \mathbf{N}$, $Q_{me}(n_j^i)$

For any $n_2^i \in \mathbf{N}$

If $L_{ins}(n_2^i) \geq L_{th}(n_2^i)$, $L_{th}(n_2^i) = \sigma L_{max}(n_2^i)$

$\breve{n}_2^i \leftarrow n_2^i \in \breve{\mathbf{N}}, \breve{\mathbf{N}} \subseteq \mathbf{N}$ \\ *Overutilized node*

$L_{exc}(n_2^i) = L_{ins}(n_2^i) - L_{th}(n_2^i)$ \\ *Excess load*

$\mathbf{M_1} = \{n_2^i | d(\breve{n}_2^i, n_2^i) = 1\}$ \\ *First tier proximity*

$L_{aval}(n_2^i) = \sigma L_{max}(n_2^i) - L_{ins}(n_2^i)$ \\ *Available load in first tier*

If $\sum_{n_2^i \in \mathbf{M}_1} L_{aval}\left(n_2^i\right) \geq L_{exc}\left(\breve{n}_2^i\right)$

Case 1: Migration to *least-load* (LL) nodes

$\mathbf{M_1} \rightarrow \mathbf{M}_1^{ll} = \{n_2^i | L_{aval}(n_2^i) \geq L_{aval}(n_2^{i+1})\}$ \\ *Resort nodes*

Case 2: Migration to *least-delay* (LD) nodes

$\mathbf{M_1} \rightarrow \mathbf{M}_1^{ld} = \{n_2^i | D(\breve{n}_2^i, n_2^i) \leq D(\breve{n}_2^i, n_2^{i+1})\}$ \\ *Resort nodes*

$D(\breve{n}_2^i, n_2^i) = D_{prp}e(\breve{n}_2^i, n_2^i) + D_{prc}(n_2^i)$

Determine amount of migrated load

If $L_{aval}\left(\mathbf{M}_1^{ld}\{1\}\right) \geq L_{exc}\left(\breve{n}_2^i\right)$ \\ *LD node possess high resource*

Single coarse-grained migration

$L_{mig}\left(\breve{n}_2^i \rightarrow \mathbf{M}_1^{ld}\{1\}\right) = L_{exc}(\breve{n}_2^i)$

Else \\ *Multiple coarse-grained migration steps*

Identify $|\mathbf{K}|$ possess sufficient available resources

$$\mathbf{K} = \left\{ n_2^i \middle| min \sum_{n_2^i \in \mathbf{M}_1} L_{aval}\left(n_2^i\right) \geq L_{exc}\left(\breve{n}_2^i\right) \right\}$$

Fragment $L_{exc}(\breve{n}_2^i)$ for migration to $|\mathbf{K}|$ only

$$L_{mig}^k\left(\breve{n}_2^i \rightarrow n_2^k\right) = \frac{L_{aval}\left(n_2^k\right)}{\sum_{k=1}^{K} L_{aval}\left(n_2^k\right)} \quad \text{\\ Load per node } k$$

Else \\ *Identify second-tier proximity*

$\mathbf{M_2} = \{n_2^i | d(\breve{n}_2^i, n_2^i) = 2\}$ \\ *Underutilized accepting nodes*

Repeat process for Case 1 or 2.

End For

Fig. 2. Load migration schemes for NFV-based fog networks

Otherwise, the excess load is partitioned into large portions for migration to the least number of neighboring nodes $|\mathbf{K}|$ of participating nodes in $\mathbf{M}_1^{ld}$,

$$\mathbf{K} = \left\{ n_2^k \middle| min \sum_{n_2^k \in \mathbf{M}_1} L_{aval}\left(n_2^k\right) \geq L_{exc}\left(\breve{n}_2^i\right) \right\}. \tag{7}$$

The fragmented excess load per each node in $\mathbf{K}$ is gauged as,

$$L_{mig}^k\left(\breve{n}_2^i \rightarrow n_2^k\right) = \frac{L_{aval}\left(n_2^k\right)}{\sum_{k=1}^{K} L_{aval}\left(n_2^k\right)}. \tag{8}$$

The coarseness process aims to perform migration using the least possible number of neighboring nodes while maintaining a utilization threshold to avoid saturation at proximity nodes. Hence, the migration process is completed in fewer iterations, contingent upon resource availability at the first and second neighboring tiers.

V. Performance Evaluation

The proposed load migration scheme for the NFV-based fog-cloud architecture is now presented for a various number of batch incoming requests. It is evaluated in terms of the migration frequency, incurred migration delay, and number of successful requests. See Table I for the complete parameters used in the network simulation. Consider the details.

TABLE I. SYSTEM SETTINGS

Network parameters	$\mathbf{N}$, $\breve{\mathbf{N}}$, $\mathbf{E}$, $Q_{prc}(n_2^i)$, $Q_{prc}(n_3^i)$, $C(n_j^i)$, $C(e)$, $D_{prc}(n_2^i)$, $D_{prc}(e)$	25, 25, 61, 60, 200, 6, 5, [0.4, 0.6] sec, 1 sec.
Request parameters	$\mathbf{R}_{map}$, $\lvert\mathbf{F}_r\rvert$, b_r, $q_{me}(f_t)$, $q_{prc}(f_t)$, T, δ_r (ms)	120, [4, 6], [4, 6], [2, 3], [2, 4], 6, 10
Load parameters	σ, $L_{max}(n_2^i)$, $L_{max}(n_3^i)$, b_e	0.7, 55, 180, 8

A. Migration Frequency

The migration frequency for the proposed offloading scheme defines the number of iterations required to diffuse a load from an overutilized node to the underutilized nodes in the proximity. The iterations scale proportionally with the excess load, available link bandwidth, and available resources at the receiving nodes. Along this, Figure 3 depicts the migration frequency for the two proposed schemes for $\mathbf{R}_{map}$ mapped requests on the fog nodes. At a small number of requests (e.g., 20), the network is at a low utilization rate and the nodes still possess abundant resources. Hence migration is not triggered at this stage. The network continues to host incoming traffic until congestion starts to appear after 50 requests, then the migration process is initiated at each cluster. Here 1-2 migration steps occur on average per node at each cluster, where VNFs are migrated regardless of their sequence in the SFC, since the migration occurs to the first or second proximity tier, i.e., SFC route is still feasible at a low incurred delay in the neighborhood of the mapped VNFs. Both the LD- and LL-based schemes perform similarly, as they perform coarse-grained migration in the least number of steps.

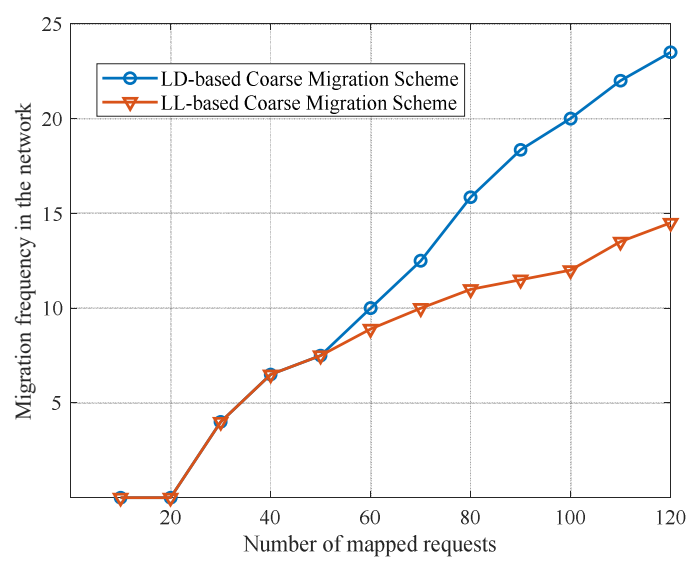

Figure 3. Migration frequency for different number of requests

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 08:42:04 UTC from IEEE Xplore. Restrictions apply.

At a high number of incoming requests, the LL scheme yields in the least migration steps to alleviate overutilized nodes as compared to the LD scheme. The LL scheme migrates first to a recipient node that possesses sufficient resources for the entire excess load. This allows for a coarse-grained diffusion without multiple fragmentations of the excess load. Meanwhile, the LD scheme migrates first to a recipient node that is at the least separation distance, i.e., least link delay, regardless of the available resources. Hence, the excess load is fragmented into larger sections of loads, which are then migrated to multiple neighboring nodes, which results in higher migration frequencies. For example, the LL-based scheme yields in 12-15 steps for 80-120 requests. This is compared to approximately 16-24 steps in the network for the LD-based scheme for the same number of requests.

B. *Incurred Migration Delay*

The mapped requests feature a stringent delay bound to meet the low latency levels of delay-sensitive applications. Along this, the migration process needs to add minimal delays that aggregate to the provisioning time (enhanced QoE). Figure 4 shows that two migration schemes feature short incurred delays (< 2.5 ms) for the first 50 requests. Intuitively, this value starts to increase proportionally with more hosted requests, where the migration delay becomes inevitable. For example, the LL-based scheme incurs 5 ms when the network is congested at 120 requests, as compared to 7.75 ms for the LD-based method.

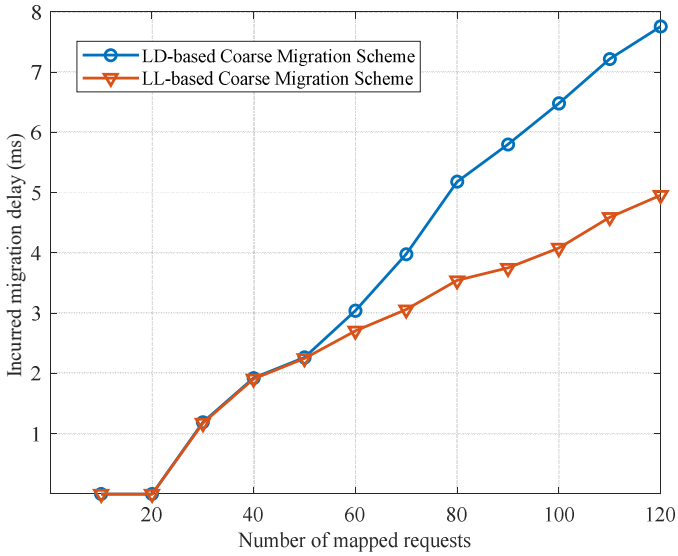

Fig. 4. Migration delay for different number of requests

The LL-based scheme allocates the first neighboring node that possesses enough resources, albeit with higher link delays. This allows for coarse-grained migration, which transfers the excess load in fewer iterations. Hence, the total link delay is still relatively low after the migration procedure is completed. On the other side, the LD-based scheme identifies the first neighboring node that yields in the least delay. However, this node probably does not possess available resources to host the entire excess load in a few migration steps. Hence the load is segmented, and multiple iterations are required to migrate the load from the overutilized node. Consequently, this yields in aggregated link and node delays that exceed the delay in the LL-based scheme. Further, the latter terminates the migration process earlier, thus occupying the network resources for reduced periods. As a result, the resources can be efficiently utilized for additional incoming requests.

C. *Migration Cost*

The migration cost refers to the additional network physical resources (nodes and links) that are added to the request path as a result of the migration process. The path still joins the *src* and *dst* nodes through a higher number of intermediate nodes, as compared to conventional embedding without load migration.

The LL-based scheme performs coarse-grained diffusion over a limited number of nodes, which is a key saliency that adds fewer costs to the network infrastructure. Here the cost is mainly attributed to the relay links that interconnect to the first *least-loaded* node in the *first* or *second* neighboring tier. For example, this scheme requires 22 and 37 units for 60 and 120 requests, respectively. This is compared to 24 and 61 units for the LD-based scheme. Here the gap between the two schemes widens proportionally with the increase of mapped requests. This is attributed to the network congestion and increased resource utilization, which reduces the number of vacant nodes. This increases network hopping to reach the destination node over multiple intermediate nodes that add to the network cost.

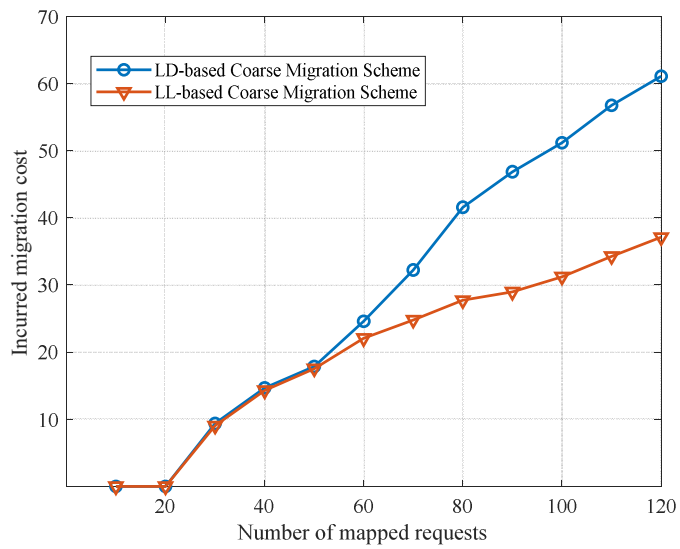

Fig. 5. Migration cost for different number of requests

Overall, the proposed migration schemes present an efficient resource management tool for service provisioning, when operating at realistic and practical network settings., e.g., node saturation and network congestion. The LD-based scheme is suitable for requests that demand high computing resources with a relaxed delay bound, at which the incurred delay (due to the fragmented excess load) remains below the delay bound of the request. Meanwhile, the LL-based scheme is more suitable for requests that demand intensive computing resources along with low delay-bounds, e.g., delay-sensitive IoT requests. Hence, coarse-grained migration occurs in fewer steps with minimal incurred delay.

VI. Conclusions

Fog computing provisions services at low latency and response times for end-user terminals. This saliency comes at the expense of limited resources at the fog nodes. This impedes fog paradigms to accommodate high traffic volumes. Further, the

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 08:42:04 UTC from IEEE Xplore. Restrictions apply.

fog nodes can saturate early if terminals demand intensive data computation. Therefore, efficient resource management becomes vital for fog computing. Along this, this paper proposes novel load migration schemes for NFV-based fog computing that diffuses the excess load from overutilized nodes to nearby fog nodes that possess available resources. The migration decision leverages *least-delay* (LD) and *least-load* (LL) approaches to adopt various types of services, e.g., delay-sensitive and delay-tolerant, and computation-intensive. Results show that the schemes yield minimal levels of migration frequencies, incurred delay, and cost for a high number of mapped requests in the network. Future efforts aim to examine migration schemes for network resiliency and survivability.

REFERENCES

[1] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the internet of things," in *Proc. 1st Edition MCC Workshop Mobile Cloud Computing*, Helsinki, Finland, Aug. 2012.

[2] M. Chiosi *et al*., "Network Functions Virtualisation: An Introduction, Benefits, Enablers, Challenges & Call for Action," ETSI White Paper, Oct. 2012.

[3] M. Jasim, N. Siasi, S. Malapaka, D. Oliveira, and O. Ugweje, "A single-tier fog architecture for delay-sensitive and computation-intensive sfc requests," in *IEEE Annual Information Technology, Electronics and Mobile Communication Conference* (*IEMCON*). Oct. 2020. Vancouver, BC, Canada.

[4] A. Jaesim, N. Siasi, M. Ababneh, M. Elkourdi, and A. Nour, "Application-specific service function chain provisioning in heterogeneous fog networks," in *IEEE International Conference on Advanced Networks and Telecommunications Systems* (*ANTS*). Dec. 2020, New Delhi, India.

[5] N. Siasi, M. Jasim, A. Aldalbahi, and N. Ghani, "Deep learning for service function chain provisioning in fog computing," *IEEE Access*, Vol. 8, pp. 167665-167683, Sept. 2020

[6] N. Siasi, M. Jasim, A. Aldalbahi and N. Ghani, "Delay-aware SFC provisioning in hybrid fog-cloud computing architectures," *IEEE Access*, Vol. 8, pp. 167383-167396, Sept. 2020.

[7] S. Ningning, G. Chao, A. Xingshuo, and Z. Qiang, "Fog computing dynamic load balancing mechanism based on graph repartitioning," *China Commun.*, Vol. 13, No. 3, pp. 156-164, Mar. 2016.

[8] D. Puthal, R. Ranjan, A, Nanda, P, Nanda, P. P. Jayaraman, and A. Y. Zomaya, "Secure authentication and load balancing of distributed edge datacenters," *Journal of Parallel and Distributed Computing* Vol. 124, pp. 60-69, Feb. 2019.

[9] Wan, Jiafu, B. Chen, S. Wang, M. Xia, D. Li, and C. Liu, "Fog computing for energy-aware load balancing and scheduling in smart factory," *IEEE Transactions on Industrial Informatics* Vol. 14, No. 10, pp. 4548-4556, Mar. 2018.

[10] P. Cai, Y. Fuqian, J. Wang, X. Wu, Y. Yang, and X. Luo. "Jote: Joint offloading of tasks and energy in fog-enabled iot networks," *IEEE Internet of Things Journal*, Vol. 7, No. 4, pp. 3067-3082, Apr. 2020.

[11] Y. Yang *et al*., "MEETS: Maximal energy efficient task scheduling in homogeneous fog networks," *IEEE Internet Things Journal*, Vol. 5, No. 5, pp. 4076–4087, Oct. 2018.

[12] J. Yang, L. Ling, and H. Liu, "A hierarchical load balancing strategy considering communication delay overhead for large-distributed computing systems," *Mathematical Problems in Engineering*, Apr. 2016.

[13] Q. Jin, L. Rongheng, H. Zou, and F. Yang, "A distributed fog computing architecture supporting multiple migrating mode," in *2018 5th IEEE International Conference on Cyber Security and Cloud Computing (CSCloud)/2018 4th IEEE International Conference on Edge Computing and Scalable Cloud (EdgeCom)*, Shanghai, China. Jul. 2018.

[14] OpenFog Consortium Architecture Working Group. OpenFog reference architecture for fog computing. OPFRA001. 2017 Feb;20817:162.

Authorized licensed use limited to: Tsinghua University. Downloaded on April 07,2026 at 08:42:04 UTC from IEEE Xplore. Restrictions apply.